

LOVELY PROFESSIONAL UNIVERSITY, PUNJAB

Project Report

Trip Mate AI – Intelligent AI Powered Travel Companion

Submitted By:

- **Name:** Ganta Sai Mani Chand
- **Registration No:** 12405533
- **Roll No:** 66
- **Section:** 3OM56

Under the Supervision of:

- **Arsalan Manzoor Zargar**

Date and Year: May 2026

- **Course:** Mobile Application Development
 - **Technology:** Android (Kotlin)
 - **Version:** 1.0
 - **GitHub Repository:** <https://github.com/SaiYadav17/TRIPMATE>
-

1. Project Overview

Trip Mate AI is a cutting-edge Android application built entirely with Kotlin and Jetpack Compose. It functions as an intelligent, AI-powered travel assistant that lets users generate complete travel itineraries using simple natural language prompts, followed by real-time open-source navigation, live GPS tracking, and location-aware recommendations.

Manually planning a trip is a tedious process that usually requires switching between multiple apps for scheduling, navigation, place ratings, and budget tracking. **Trip Mate AI** integrates all of these processes into a single, cohesive experience. It uses the Gemini 1.5 Pro/Flash models to create a structured day-by-day plan with coordinates, parses those coordinates to render a live vector map using **OSMDroid**, queries the **OSRM Routing Engine** to construct live navigation lines, and continuously queries the **Overpass API** to spot nearby scenic viewpoints, cafes, and restaurants along the route.

Best of all, the entire application relies strictly on free, open-source mapping services, keeping the operational cost completely free for the user.

2. Module-Wise Breakdown

2.1 Splash & Onboarding Module

Upon application launch, the `SplashScreen` plays a dynamic radial gradient background animation with pulsing loading indicators for 2.5 seconds. After fading in the logo, the app navigates to the `OnboardingScreen`. The onboarding module features 3 swipe-friendly cards containing beautifully crafted illustrative summaries of the app's value proposition:

- **AI Trip Planner:** Explaining natural language prompt itinerary construction.
- **Live Navigation:** Highlighting open-source maps and zero-cost navigation.
- **Smart Suggestions:** Outlining automatic place detection and scenic viewpoints.

2.2 AI Assistant Module (Home Screen)

The home screen utilizes a premium glassmorphic input card where users specify their travel parameters (e.g., *"Plan a 3-day Goa trip under ₹10,000"*).

- Pre-filled interactive prompt chips allow one-tap testing.
- A custom Lottie-inspired "AI Thinking" loading animation indicates content generation.
- The module packages prompts with strong system guidelines, ensuring the Gemini model replies only in pure JSON that complies with the schema.

2.3 Itinerary Screen Module

The generated plan is parsed into the custom `TripPlan` structure and presented as structured cards:

- **Day-wise Tabbed Selector:** Animated M3 tab rows to switch between Days.
- **Place Cards:** Rendered using slide-in animations. Cards display category emojis (🏛️, 🍴, 🌳), time durations, and budget estimates.
- **Aggregated Lists:** Highlights, packing lists, and specific meal recommendations.
- **Action FABs:** Double floating buttons for launching real-time navigation or opening the conversational companion.

2.4 Map & Navigation Module

This is the core mapping engine utilizing **OSMDroid**. It renders high-quality map tiles inside a Compose `AndroidView`.

- **Live GPS Layer:** Visualizes user location with custom marker anchoring.
- **Route Polyline Drawing:** Translates OSRM coordinates into `GeoPoint` segments drawn dynamically on the map canvas.
- **Navigation Overlay:** Displays step-by-step turn-by-turn driving steps (e.g., *"Turn left onto NH66"*), remaining distance, and ETA calculations.

2.5 AI Chat Assistant Module

A contextual chat interface allowing users to converse directly with their travel companion. When the chat screen opens, it passes the entire itinerary structure behind the scenes to Gemini. The user can ask, *"Where should I go for dinner tonight?"* or *"What is a good packing tip for this trip?"* and get highly context-aware responses with typing indicators.

2.6 Nearby suggestions Module

Utilizing the **Overpass API**, the app calculates the distance traveled from the last API call. Once the user moves more than 500 meters, it queries OpenStreetMap node clusters for nearby amenities (tourism viewpoints, restaurants, cafes, etc.) and lists them on a bottom scrolling carousel.

2.7 Theme & UI System

Custom high-fidelity theme styling defined in `Theme.kt`:

- **Futuristic Dark Palette:** High contrast neon accents (`#6C63FF` Electric Violet, `#00D4FF` Cyan Accent, and `#0A0E1A` Deep Navy).
 - **Glassmorphism:** Visual card surfaces utilizing white transparencies with linear gradient borders and shadow blurs.
-

3. Functionalities

1. **User Welcome Onboarding:** Swipeable multi-slide onboarding with skip/next logic.
2. **AI-Driven Travel Prompts:** Accepts natural language prompt input.
3. **Structured Itinerary Parsing:** Converts unformatted AI generation into robust typed Kotlin structures.
4. **Day-wise Segmented Plans:** Fast day tabs for simple viewing.
5. **Interactive Place Cards:** Custom-designed cards categorized with dedicated icons and durations.
6. **Live OSM Map Integration:** Renders full-screen interactive OpenStreetMap view.
7. **OSRM Route Generation:** Requests and renders exact driving routes between the current location and destination.
8. **Continuous GPS Tracking:** Monitors user coordinates using the Android Fused Location provider.
9. **Turn-by-Turn Navigation:** Displays textual direction instructions updated in real-time.
10. **ETA & Distance Computations:** Shows remaining distance in km/meters and travel time in hours/mins.
11. **Overpass Nearby POI Discovery:** Scans 1km radius for landmarks, restaurants, and scenic spots automatically.
12. **Location-Aware Suggestions:** Re-calculates and fetches new POIs once the user moves 500m.
13. **Contextual AI Chatbot:** Interactive chat screen with access to current trip context.
14. **Quick Chat Chips:** Fast prompts (e.g., *"Save budget"*, *"What to eat"*) for ease of access.
15. **Typing Indicators:** Custom bouncing-dots loader indicating AI response streaming.
16. **Dynamic Packing Lists:** Automatically provides packing suggestions per trip.
17. **Dynamic Travel Tips:** Displays destination-specific guidelines.
18. **Custom Glassmorphic Cards:** High-fidelity M3 components using translucent backdrops.
19. **Bouncing Animations:** Smooth scaling, fading, and micro-interactions on buttons.
20. **Network Error Handling:** Integrated fallback states when the user goes offline.
21. **Auto-centering Map Camera:** Auto-pans map coordinates as the user travels.
22. **Interactive Markers:** Tap markers to reveal spot summaries.

- 23. **Mock Navigation Triggering:** Test navigation endpoints from anywhere without needing GPS.
 - 24. **Cleartext Map Traffic:** Configured custom XML network security to support smooth OSM tile downloading.
 - 25. **Fully Free Map Engine:** Operating cost is zero, omitting standard Google Maps API billing.
-

4. Technology Used

4.1 Programming Languages

- **Kotlin:** 100% of the logical layer, database interactions, ViewModels, and Compose layouts.
- **Kotlin DSL:** Gradle script build specifications.
- **XML:** Android Manifest and network clearance protocols.

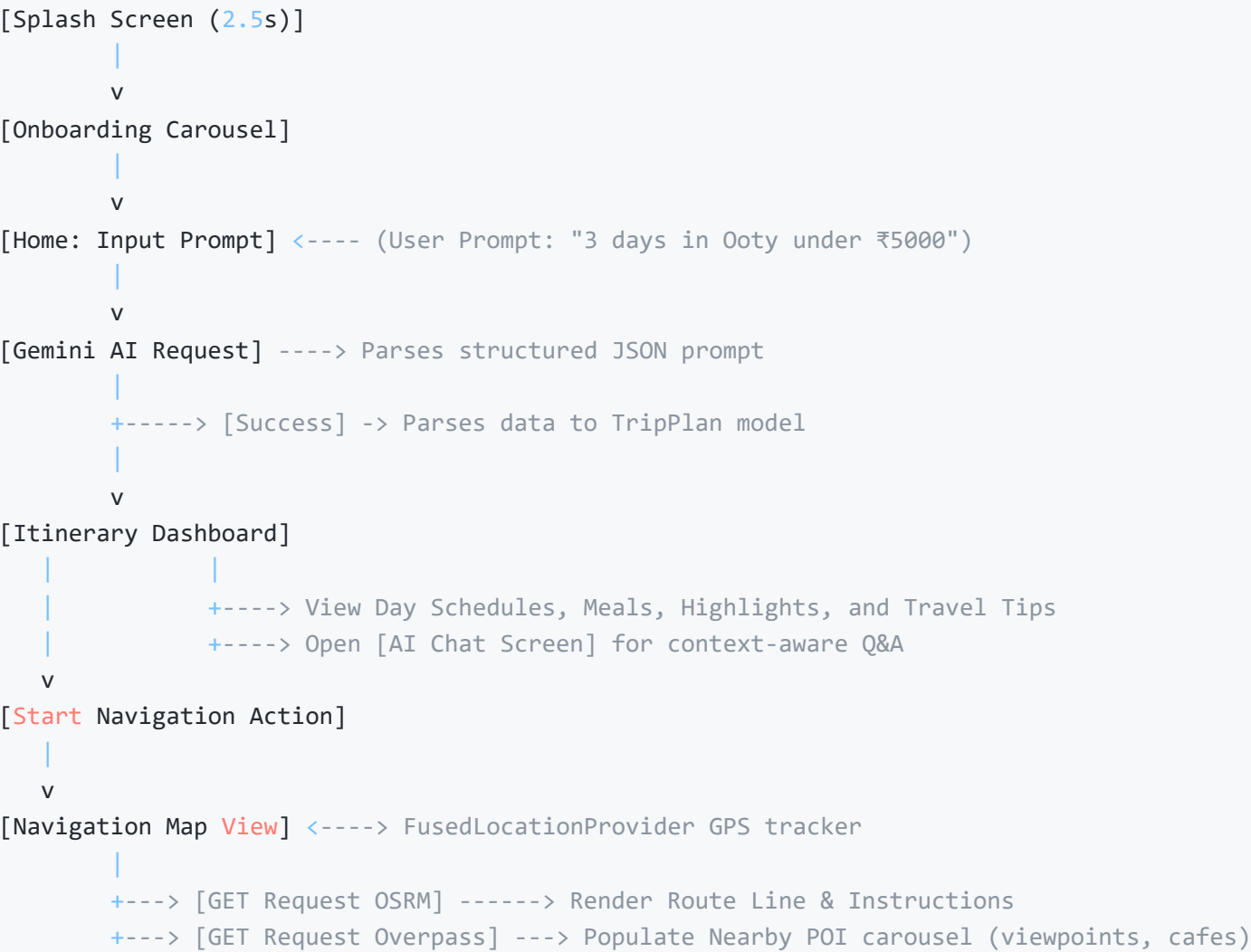
4.2 Frameworks & SDKs

- **Jetpack Compose:** Declared modern reactive design system.
- **Android SDK:** Built with target API level 34.

4.3 Libraries and Dependencies

Library	Version	Purpose
OSMDroid Android	6.1.18	Open-source map layer container and rendering engine
Google Generative AI SDK	0.7.0	Seamless Gemini 1.5 Flash API connection
Retrofit 2	2.9.0	Decoupled HTTP interface definitions for OSRM & Overpass APIs
Gson Converter	2.9.0	Automatic JSON serialization and deserialization
Play Services Location	21.2.0	Android FusedLocationProvider access for high-precision GPS
Navigation Compose	2.7.7	Single Activity Navigation Controller Graph
OkHttp Logging Interceptor	4.12.0	Detailed network log inspection in Debug console
Lifecycle Viewmodel Compose	2.8.0	Scope-aware ViewModels and StateFlow integrations

5. Flow Diagram



6. GitHub Repository Revision Tracking

- **Repository:** TRIPMATE
 - **GitHub Link:** <https://github.com/SaiYadav17/TRIPMATE>
 - **Organization & Pattern:** Follows clean architecture split:
 - /data/model/ : Serialization schemas.
 - /data/remote/ : Retrofit API endpoint contracts.
 - /data/repository/ : Network and location provider data layers.
 - /ui/theme/ and /ui/components/ : Modular Compose widgets and system color styles.
 - /ui/screens/ : Splitted screen files.
 - /viewmodel/ : Separate controllers for prompt handling, map calculations, and chatbot flows.
-

7. Conclusion and Future Scope

7.1 Conclusion

Trip Mate AI successfully demonstrates the capability of open-source frameworks in replacing expensive commercial mapping services. By integrating Gemini AI and OSMDroid, we have established a highly interactive mobile workspace. The modular architecture facilitates quick maintenance and ensures state mutations are cleanly separated via ViewModel flows.

7.2 Future Scope

- **Offline Map Support:** Pre-caching vector tiles for offline travel navigation.
 - **Group Coordination:** Real-time web socket integrations to share active navigation paths between friends.
 - **Voice Navigations:** Text-to-speech directions for hand-free driver guidance.
 - **Local Currency Conversions:** Automatic dynamic pricing computations using active exchange rate APIs.
-

8. References

1. **Jetpack Compose Guides:** <https://developer.android.com/jetpack/compose>
 2. **OSMDroid Integration Wiki:** <https://github.com/osmdroid/osmdroid>
 3. **Gemini Android SDK:** <https://ai.google.dev/gemini-api/docs/quickstart/android>
 4. **OSRM Server Documentation:** <http://project-osrm.org/>
 5. **Overpass Query Language:** https://wiki.openstreetmap.org/wiki/Overpass_API
-

Appendix A – Architecture & Core States

The application adheres to clean MVVM (Model-View-ViewModel) architecture:

- **Models:** Strongly typed representations of data structures (`TripPlan.kt` , `NearbyPlace.kt`).
 - **Views:** Jetpack Compose screens that react immediately to exposed states.
 - **ViewModels:** Reactive managers (`HomeViewModel.kt` , `NavigationViewModel.kt` , `ChatViewModel.kt`) tracking state changes through Kotlin `StateFlow` .
-

Appendix B – Source Code Listings

Below is the implementation source code of the **Trip Mate AI** mobile application.

B.1 MainActivity.kt

Entry point managing the single-activity architecture, edge-to-edge layout, and the theme wrapper.

```
package com.tripmate.ai

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Surface
import androidx.compose.ui.Modifier
import androidx.navigation.compose.rememberNavController
import com.tripmate.ai.navigation.TripMateNavGraph
import com.tripmate.ai.ui.theme.DeepNavy
import com.tripmate.ai.ui.theme.TripMateTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            TripMateTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = DeepNavy
                ) {
                    val navController = rememberNavController()
                    TripMateNavGraph(navController = navController)
                }
            }
        }
    }
}
```

B.2 NavGraph.kt

State-driven Single Activity navigation graph with type-safe screen transition arguments.


```
package com.tripmate.ai.navigation

import androidx.compose.runtime.*
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavHostController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import com.tripmate.ai.data.model.TripPlan
import com.tripmate.ai.ui.screens.chat.ChatScreen
import com.tripmate.ai.ui.screens.home.HomeScreen
import com.tripmate.ai.ui.screens.itinerary.ItineraryScreen
import com.tripmate.ai.ui.screens.navigation.NavigationScreen
import com.tripmate.ai.ui.screens.onboarding.OnboardingScreen
import com.tripmate.ai.ui.screens.splash.SplashScreen
import com.tripmate.ai.viewmodel.HomeViewModel

object Routes {
    const val SPLASH = "splash"
    const val ONBOARDING = "onboarding"
    const val HOME = "home"
    const val ITINERARY = "itinerary"
    const val NAVIGATION = "navigation/{lat}/{lon}/{name}"
    const val CHAT = "chat"

    fun navigation(lat: Double, lon: Double, name: String) =
        "navigation/$lat/$lon/${name.replace("/", "-")}"
}

@Composable
fun TripMateNavGraph(navController: NavHostController) {
    val homeViewModel: HomeViewModel = viewModel()

    NavHost(navController = navController, startDestination = Routes.SPLASH) {
        composable(Routes.SPLASH) {
            SplashScreen(
                onFinish = {
                    navController.navigate(Routes.ONBOARDING) {
                        popUpTo(Routes.SPLASH) { inclusive = true }
                    }
                }
            )
        }

        composable(Routes.ONBOARDING) {
            OnboardingScreen(
                onGetStarted = {
                    navController.navigate(Routes.HOME) {

```

```

        popUpTo(Routes.ONBOARDING) { inclusive = true }
    }
}

)
}

composable(Routes.HOME) {
    HomeScreen(
        viewModel = homeViewModel,
        onTripGenerated = { navController.navigate(Routes.ITINERARY) }
    )
}

composable(Routes.ITINERARY) {
    val tripPlan = homeViewModel.uiState.collectAsState().value.tripPlan ?: TripPlan()
    ItineraryScreen(
        tripPlan = tripPlan,
        onStartNavigation = { lat, lon, name ->
            navController.navigate(Routes.navigation(lat, lon, name))
        },
        onOpenChat = { navController.navigate(Routes.CHAT) },
        onBack = { navController.popBackStack() }
    )
}

composable(
    route = Routes.NAVIGATION,
    arguments = listOf(
        androidx.navigation.navArgument("lat") { type = androidx.navigation.NavType.String },
        androidx.navigation.navArgument("lon") { type = androidx.navigation.NavType.String },
        androidx.navigation.navArgument("name") { type = androidx.navigation.NavType.String }
    )
) { backStackEntry ->
    val lat = backStackEntry.arguments?.getString("lat")?.toDoubleOrNull() ?: 0.0
    val lon = backStackEntry.arguments?.getString("lon")?.toDoubleOrNull() ?: 0.0
    val name = backStackEntry.arguments?.getString("name") ?: "Destination"
    NavigationScreen(
        destLat = lat,
        destLon = lon,
        destName = name,
        onBack = { navController.popBackStack() }
    )
}

composable(Routes.CHAT) {
    val tripPlan = homeViewModel.uiState.collectAsState().value.tripPlan ?: TripPlan()
    ChatScreen(
        tripPlan = tripPlan,

```

```
        onBack = { navController.popBackStack() }  
    )  
}  
}
```

B.3 TripPlan.kt

Data schemas representing parsed travel structures and category icons mapping.

```

package com.tripmate.ai.data.model

data class TripPlan(
    val destination: String = "",
    val totalDays: Int = 0,
    val totalBudget: String = "",
    val highlights: List<String> = emptyList(),
    val days: List<DayPlan> = emptyList(),
    val travelTips: List<String> = emptyList(),
    val packingList: List<String> = emptyList()
)

data class DayPlan(
    val day: Int = 1,
    val theme: String = "",
    val places: List<PlaceItem> = emptyList(),
    val meals: List<String> = emptyList(),
    val estimatedDailyCost: String = "",
    val tips: String = ""
)

data class PlaceItem(
    val name: String = "",
    val description: String = "",
    val category: String = "attraction",
    val estimatedTime: String = "",
    val estimatedCost: String = "",
    val lat: Double = 0.0,
    val lon: Double = 0.0
) {
    fun categoryIcon(): String = when (category.lowercase()) {
        "restaurant", "food" -> "🍴"
        "hotel", "accommodation" -> "🏨"
        "viewpoint", "scenic" -> "🌄"
        "cafe" -> "☕"
        "reststop", "rest_stop" -> "🛑"
        "beach" -> "🏖️"
        "temple", "religious" -> "🕌"
        "museum" -> "🏛️"
        "park", "nature" -> "🌿"
        "shopping" -> "🛍️"
        else -> "📍"
    }
}

data class ChatMessage(
    val content: String,

```

```
val isUser: Boolean,  
val timestamp: Long = System.currentTimeMillis()  
)
```

B.4 GeminiRepository.kt

Core AI generation interface containing advanced prompt engineering, JSON response extraction, and parsing fallbacks.

```
package com.tripmate.ai.data.repository

import com.google.ai.client.generativeai.GenerativeModel
import com.google.ai.client.generativeai.type.generationConfig
import com.google.gson.Gson
import com.google.gson.JsonSyntaxException
import com.tripmate.ai.BuildConfig
import com.tripmate.ai.data.model.TripPlan
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow

sealed class GeminiResult {
    data class Success(val tripPlan: TripPlan) : GeminiResult()
    data class Error(val message: String) : GeminiResult()
    object Loading : GeminiResult()
}

sealed class ChatResult {
    data class Success(val text: String) : ChatResult()
    data class Error(val message: String) : ChatResult()
    object Loading : ChatResult()
}

class GeminiRepository {
    private val gson = Gson()

    private val model = GenerativeModel(
        modelName = "gemini-1.5-flash",
        apiKey = BuildConfig.GEMINI_API_KEY,
        generationConfig = generationConfig {
            temperature = 0.7f
            maxOutputTokens = 4096
        }
    )

    private val chatModel = GenerativeModel(
        modelName = "gemini-1.5-flash",
        apiKey = BuildConfig.GEMINI_API_KEY,
        generationConfig = generationConfig {
            temperature = 0.9f
            maxOutputTokens = 1024
        }
    )

    fun generateTripPlan(prompt: String): Flow<GeminiResult> = flow {
        emit(GeminiResult.Loading)
        try {
```

```

        val structuredPrompt = buildPrompt(prompt)
        val response = model.generateContent(structuredPrompt)
        val rawText = response.text ?: throw Exception("Empty response from AI")

        val jsonText = extractJson(rawText)
        val tripPlan = gson.fromJson(jsonText, TripPlan::class.java)
        emit(GeminiResult.Success(tripPlan))
    } catch (e: JsonSyntaxException) {
        emit(GeminiResult.Error("Failed to parse trip plan. Please try again. "))
    } catch (e: Exception) {
        emit(GeminiResult.Error(e.message ?: "Something went wrong. Please try again. "))
    }
}

suspend fun askAboutTrip(question: String, tripContext: String): Flow<ChatResult> = flow {
    emit(ChatResult.Loading)
    try {
        val contextualPrompt = """
            You are Trip Mate AI, a helpful travel assistant.
            The user is currently on this trip:
            $tripContext

            User question: $question

            Provide a helpful, concise, and friendly response. Keep it under 150 words.
            """.trimIndent()

        val response = chatModel.generateContent(contextualPrompt)
        emit(ChatResult.Success(response.text ?: "I'm sorry, I couldn't generate a response. "))
    } catch (e: Exception) {
        emit(ChatResult.Error(e.message ?: "Failed to get AI response. "))
    }
}

private fun buildPrompt(userPrompt: String): String = """
    You are an expert travel planner. Generate a detailed travel itinerary based on: "$userPrompt"

    Return ONLY valid JSON (no markdown, no extra text) in this exact format:
    {
        "destination": "City/Place Name",
        "totalDays": 3,
        "totalBudget": "₹15,000",
        "highlights": ["highlight1", "highlight2", "highlight3"],
        "days": [
            {
                "day": 1,
                "theme": "Arrival & Exploration",
                "places": [

```

```

        {
            "name": "Place Name",
            "description": "2-3 sentence description",
            "category": "attraction",
            "estimatedTime": "2 hours",
            "estimatedCost": "₹200",
            "lat": 15.4909,
            "lon": 73.8278
        }
    ],
    "meals": ["Breakfast: Suggestion", "Lunch: Suggestion", "Dinner: Suggestion"],
    "estimatedDailyCost": "₹3,000",
    "tips": "Helpful tip for the day"
}
],
"travelTips": ["tip1", "tip2", "tip3"],
"packingList": ["item1", "item2", "item3"]
}

```

Category must be one of: attraction, restaurant, hotel, viewpoint, cafe, reststop, beach
 Include real GPS coordinates (lat/lon) for each place.

```
"".trimIndent()
```

```

private fun extractJson(text: String): String {
    val cleaned = text.replace("`json", "").replace("`", "").trim()
    val start = cleaned.indexOf('{')
    val end = cleaned.lastIndexOf('}')
    return if (start != -1 && end != -1) cleaned.substring(start, end + 1) else cleaned
}
}

```

B.5 NavigationScreen.kt

Combines OSMDroid Android View wrapping, Route overlay logic, and location updates for real-time turn calculations.


```
package com.tripmate.ai.ui.screens.navigation

import android.graphics.Paint
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.LocationOn
import androidx.compose.material.icons.filled.Place
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.lifecycle.viewmodel.compose.viewModel
import com.tripmate.ai.ui.components.GlassCard
import com.tripmate.ai.ui.components.NearbyPlaceChip
import com.tripmate.ai.ui.components.ShimmerBox
import com.tripmate.ai.ui.theme.*
import com.tripmate.ai.viewmodel.NavigationViewModel
import org.osmdroid.tileprovider.tilesource.TileSourceFactory
import org.osmdroid.util.GeoPoint
import org.osmdroid.views.MapView
import org.osmdroid.views.overlay.Marker
import org.osmdroid.views.overlay.Polyline

@Composable
fun NavigationScreen(
    destLat: Double,
    destLon: Double,
    destName: String,
    onBack: () -> Unit,
    viewModel: NavigationViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    var mapView by remember { mutableStateOf<MapView?>(null) }
```

```
LaunchedEffect(destLat, destLon) {
    viewModel.setDestination(destLat, destLon, destName)
    viewModel.startNavigation()
}

DisposableEffect(Unit) {
    onDispose { viewModel.stopNavigation(); mapView?.onDetach() }
}

LaunchedEffect(uiState.currentLocation, uiState.routeInfo) {
    val map = mapView ?: return@LaunchedEffect
    map.overlays.clear()

    val location = uiState.currentLocation
    if (location != null) {
        val userPoint = GeoPoint(location.latitude, location.longitude)
        val userMarker = Marker(map).apply {
            position = userPoint
            title = "You are here"
            setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM)
        }
        map.overlays.add(userMarker)
        map.controller.animateTo(userPoint)
    }

    if (destLat != 0.0) {
        val destMarker = Marker(map).apply {
            position = GeoPoint(destLat, destLon)
            title = destName
            setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM)
        }
        map.overlays.add(destMarker)
    }

    val route = uiState.routeInfo
    if (route != null && route.points.isNotEmpty()) {
        val polyline = Polyline(map).apply {
            setPoints(route.points.map { GeoPoint(it.first, it.second) })
            outlinePaint.color = ElectricViolet.toArgb()
            outlinePaint.strokeWidth = 10f
        }
        map.overlays.add(polyline)
    }
    map.invalidate()
}

Box(modifier = Modifier.fillMaxSize().background(DeepNavy)) {
    AndroidView(
```

```

        factory = { ctx ->
            MapView(ctx).also { mv ->
                mv.setTileSource(TileSourceFactory.MAPNIK)
                mv.setMultiTouchControls(true)
                mv.controller.setZoom(15.0)
                if (destLat != 0.0) mv.controller.setCenter(GeoPoint(destLat, destLon))
                mapView = mv
            }
        },
        modifier = Modifier.fillMaxSize()
    )

    GlassCard(
        modifier = Modifier.fillMaxWidth().padding(16.dp).align(Alignment.TopStart),
        borderRadius = 18.dp
    ) {
        Row(
            modifier = Modifier.padding(horizontal = 12.dp, vertical = 10.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(10.dp)
        ) {
            IconButton(onClick = onBack, modifier = Modifier.size(36.dp)) {
                Icon(Icons.Filled.ArrowBack, "Back", tint = TextPrimary)
            }
            Icon(Icons.Filled.Place, contentDescription = null, tint = ElectricViolet, mod:
            Text(
                destName,
                color = TextPrimary,
                fontWeight = FontWeight.SemiBold,
                fontSize = 15.sp,
                modifier = Modifier.weight(1f),
                maxLines = 1
            )
        }
    }

    Column(
        modifier = Modifier.fillMaxWidth().align(Alignment.BottomCenter)
    ) {
        if (uiState.isLoadingNearby) {
            LazyRow(
                contentPadding = PaddingValues(horizontal = 16.dp),
                horizontalArrangement = Arrangement.spacedBy(10.dp),
                modifier = Modifier.padding(bottom = 8.dp)
            ) {
                items(3) { ShimmerBox(modifier = Modifier.size(width = 130.dp, height = 80
            }
        } else if (uiState.nearbyPlaces.isNotEmpty()) {

```

```

Text(
    "📍 Nearby Suggestions",
    color = TextPrimary,
    fontSize = 13.sp,
    fontWeight = FontWeight.SemiBold,
    modifier = Modifier.padding(start = 16.dp, bottom = 6.dp)
)
LazyRow(
    contentPadding = PaddingValues(horizontal = 16.dp),
    horizontalArrangement = Arrangement.spacedBy(10.dp),
    modifier = Modifier.padding(bottom = 8.dp)
) {
    items(uiState.nearbyPlaces) { place ->
        NearbyPlaceChip(
            emoji = place.displayType().split(" ").first(),
            name = place.name,
            type = place.displayType()
        )
    }
}

GlassCard(
    modifier = Modifier.fillMaxWidth().padding(horizontal = 16.dp).padding(bottom = 8.dp).padding(top = 8.dp).cornerRadius = 20.dp
) {
    Column(modifier = Modifier.padding(20.dp)) {
        if (uiState.isLoadingRoute) {
            Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.Center) {
                CircularProgressIndicator(color = ElectricViolet, modifier = Modifier.size(24.dp))
                Text("Calculating route...", color = TextSecondary, fontSize = 14.sp)
            }
        } else {
            val route = uiState.routeInfo
            if (route != null) {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween
                ) {
                    Column {
                        Text("Distance", color = TextSecondary, fontSize = 12.sp)
                        Text(route.distanceText(), color = TextPrimary, fontSize = 12.sp)
                    }
                    Column(horizontalAlignment = Alignment.End) {
                        Text("ETA", color = TextSecondary, fontSize = 12.sp)
                        Text(route.durationText(), color = CyanAccent, fontSize = 12.sp)
                    }
                }
            }
        }
    }
}

```

